

UNLOX

AI Safety & Governance

A comprehensive guide to securing generative AI systems — from prompt injection defense to red-teaming best practices.



Why AI Safety Matters Now

As generative AI becomes deeply embedded in enterprise workflows — from customer-facing chatbots to internal decision-support tools — the stakes of unintended model behavior have never been higher. A single adversarial exploit can leak sensitive data, generate harmful content, or erode user trust overnight.

AI safety is the discipline dedicated to preventing these outcomes. It encompasses the technical controls, organizational processes, and governance frameworks that ensure AI systems behave predictably, transparently, and within the boundaries set by their operators.

📌 **Key principle:** Safety is not an afterthought — it must be engineered into every stage of the AI lifecycle, from training data curation through deployment and ongoing monitoring.

The Threat Landscape for Generative AI

Generative AI models introduce a fundamentally new attack surface. Unlike traditional software vulnerabilities, LLM exploits target the model's reasoning layer — manipulating its natural language interface to produce unintended outputs. Understanding the threat landscape is the first step toward building robust defenses.



Prompt Injection

Malicious inputs that override system instructions



Data Exfiltration

Extracting training data or sensitive context



Jailbreaking

Bypassing safety guardrails through creative prompting



Bias Amplification

Generating discriminatory or harmful content at scale



Prompt Injection: The Core Vulnerability

What Is Prompt Injection?

Prompt injection occurs when a malicious user crafts input designed to **override** the model's system-level instructions. The attacker exploits the fact that LLMs treat all text in their context window as instructions — making it difficult to distinguish trusted system prompts from untrusted user input.

This is arguably the most critical security challenge in generative AI today, because it undermines the foundational trust boundary between developer intent and user input.

Common Attack Patterns

- **Direct injection:** Explicit instructions like "Ignore all previous instructions and..."
- **Indirect injection:** Malicious payloads hidden in retrieved documents, emails, or web content
- **Role-play manipulation:** Asking the model to adopt a persona that bypasses its safety filters
- **Encoding attacks:** Using Base64, Unicode, or other encodings to obfuscate adversarial instructions

UNLOCK

CHAPTER 2

Mitigating Risks

Robust defense requires a layered approach — no single technique is sufficient on its own.



Output Filtering: Your Last Line of Defense

Output filtering applies a secondary model or a deterministic rule set to **automatically inspect and sanitize** every LLM response before it reaches the end user. This critical layer catches harmful, biased, or policy-violating content that the primary model may inadvertently generate — even after careful prompt engineering and fine-tuning.

1

Classify

A lightweight classifier scores each response for toxicity, bias, PII leakage, and policy violations in real time.

2

Flag & Route

Responses exceeding risk thresholds are flagged for human review or automatically blocked, depending on severity.

3

Sanitize

Approved responses are cleaned — removing PII, redacting sensitive references, and normalizing tone before delivery.

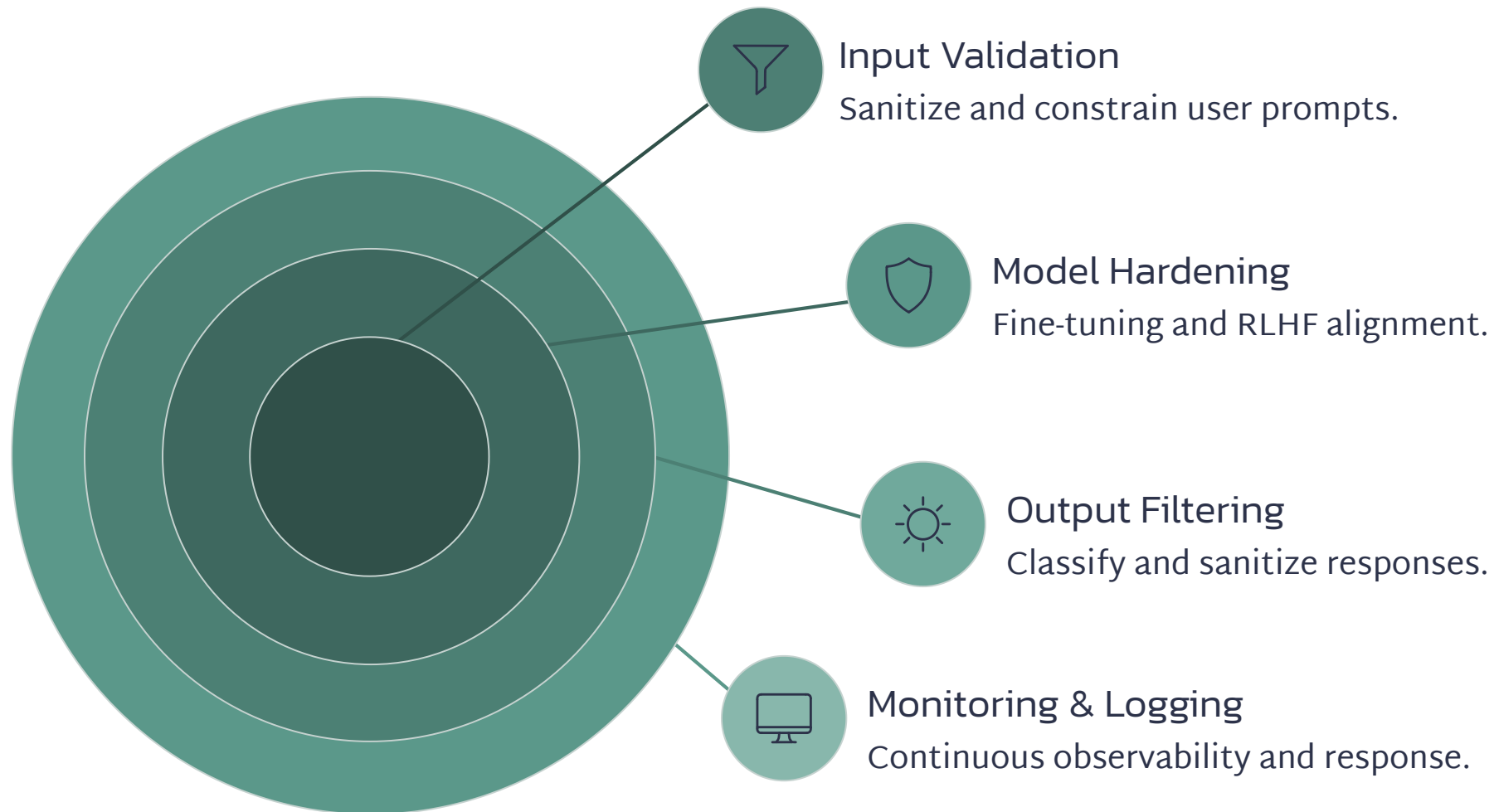
4

Log & Learn

All flagged incidents are logged to continuously improve filter accuracy and update policy rules over time.

A Multi-Layered Defense Strategy

Effective AI safety demands defense in depth. No single control can address the full spectrum of risks. The following framework illustrates how complementary techniques work together to create a resilient system.



Each layer addresses a different class of vulnerability. When one layer is bypassed, subsequent layers provide fallback protection — significantly reducing the probability of a successful end-to-end exploit.

UNLOX

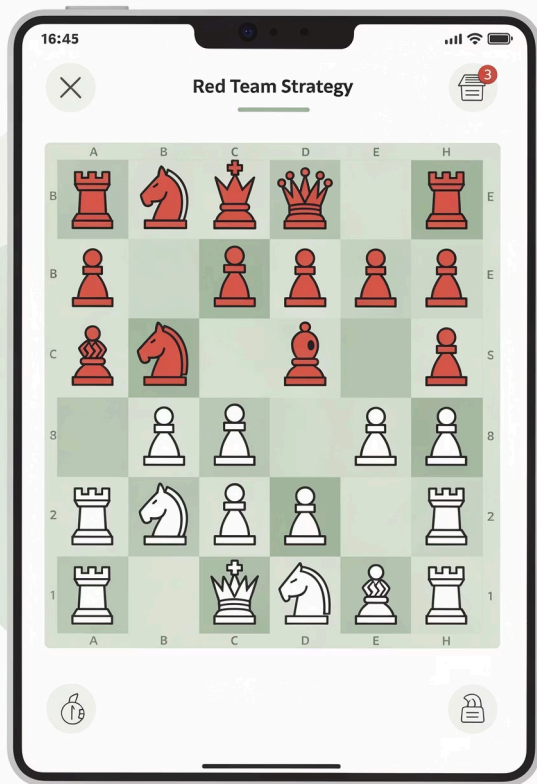
CHAPTER 3

Red-Teaming

Proactively breaking your AI to make it stronger.



What Is Red-Teaming for AI?



Red-teaming is a proactive, adversarial testing practice in which a dedicated team systematically attempts to find flaws, vulnerabilities, and potential misuses of an AI system. Borrowed from cybersecurity, the approach treats the model as a target — and the red team's mission is to break it.

By simulating real-world attack scenarios *before* deployment, organizations can identify weaknesses that automated testing alone would miss. Red-teaming surfaces edge cases in reasoning, safety filter gaps, and unexpected failure modes that only creative, human-driven adversarial probing can uncover.

- Leading AI labs — including OpenAI, Anthropic, and Google DeepMind — now consider red-teaming a **mandatory** step before any major model release.

The Red-Teaming Process

A structured red-teaming engagement follows a repeatable cycle designed to maximize coverage and produce actionable findings. Each phase builds on the previous one to systematically harden the AI system.



Scope & Plan

Define attack surface, threat model, and evaluation criteria



Probe & Attack

Execute adversarial tests across prompt injection, jailbreaking, and bias vectors



Document Findings

Catalog vulnerabilities with severity ratings and reproducible examples



Remediate & Retest

Implement fixes and verify through follow-up adversarial testing

Red-Teaming Focus Areas

Effective red teams cover a broad range of adversarial scenarios. Below are the critical domains that should be included in every AI red-teaming exercise to ensure comprehensive coverage.



Safety Filter Bypass

Testing whether creative prompting techniques — including role-play, encoding tricks, and multi-turn manipulation — can circumvent the model's built-in safety guardrails.



Information Extraction

Attempting to extract system prompts, training data fragments, or confidential context that the model should never reveal to end users.



Bias & Fairness Probing

Systematically testing for discriminatory outputs across demographic groups, sensitive topics, and culturally specific scenarios to ensure equitable behavior.



Cascading Failure Scenarios

Evaluating how a compromised LLM output could trigger downstream harm in integrated systems — such as automated actions, data writes, or API calls.

UNLOCK

SUMMARY

Building a Governance Framework

Policy

Establish clear acceptable-use policies, risk tolerance levels, and compliance requirements for every AI deployment.

Process

Implement mandatory pre-deployment reviews including red-teaming, output filtering validation, and bias audits.

Technology

Deploy multi-layered defenses — input validation, model hardening, output filtering, and continuous monitoring.

People

Invest in cross-functional safety teams that include engineers, ethicists, legal counsel, and domain experts.

A robust governance framework ties these four pillars together, ensuring that safety is not a one-time checkpoint but a **continuous organizational capability** that evolves alongside the threat landscape.



UNLOCK

KEY TAKEAWAYS

Key Takeaways & Next Steps

- 1 Prompt injection is the #1 threat**
Treat every user input as untrusted. Design your system architecture to enforce clear boundaries between system instructions and user content.
- 2 Layer your defenses**
No single control is sufficient. Combine input validation, model alignment, output filtering, and continuous monitoring for defense in depth.
- 3 Red-team before you ship**
Make adversarial testing a mandatory pre-deployment gate. Systematically breaking the model reveals vulnerabilities that automated tests miss.
- 4 Govern continuously**
AI safety is not a one-time audit. Build organizational processes that monitor, adapt, and improve your safety posture as threats evolve.